ARPE — Reference Paper Digests

Abstract, System Architecture, Results, Conclusion, and References

Team Details

S.no	Name	Register No.	Department
1	Kithiyon	ACS23003	CSE (Cybersecurity)
2	Mohamed Abdul Baasid	ACS23004	CSE (Cybersecurity)
3	Rahsheetha	ACS23007	CSE (Cybersecurity)

Reference Paper 1

BRIDG-ICS: AI-Grounded Knowledge Graphs for Intelligent Threat Analytics in Industry 5.0 Cyber-Physical Systems

Nandiya, Mohsin, Ibrahim, Sarker, Janicke — Springer, Cybersecurity journal (2026) / arXiv:2512.12112 — Edith Cowan University

Abstract

As Industry 5.0 deepens the connection between IT and OT systems, industrial control systems face a widening attack surface that legacy, siloed defenses can't address coherently. The authors present BRIDG-ICS, an AI-driven knowledge graph framework that fuses industrial and cybersecurity data — assets, vulnerabilities, and adversary behaviors — into a single graph enriched with probabilistic risk metrics such as exploit likelihood and attack cost. Large language models are used to extract entities, predict relationships, and convert plain-language threat descriptions into structured graph data, filling in associations the source datasets leave missing. The result supports multi-hop, causality-aware reasoning over attack chains. Testing on simulated industrial scenarios showed the framework scales well and measurably reduces modeled attack exposure.

System Architecture

BRIDG-ICS is built around a five-part ontology that links industrial assets to attacker behavior through a hierarchy: ICS → CVE → CWE → CAPEC → MITRE ATT&CK. Each layer is its own sub-ontology (ICS assets/protocols/zones; CVE vulnerability records enriched with EPSS/KEV/CPE data; CWE weaknesses; CAPEC attack patterns; and ATT&CK tactics/techniques), and a local industrial testbed (following the ISA-95/Purdue architecture) supplies real PLC, SCADA/HMI, and network-zone metadata to ground the graph in a concrete environment.

Graph construction happens in three steps — preprocessing the public datasets (dropping rejected CVEs, standardizing text, mapping products to CPE identifiers), integrating the local testbed's communication and operational logs via a COMMUNICATES_WITH relation, and merging everything into a node.csv/relation.csv pair loaded into Neo4j.

On top of the graph sits a risk-modeling layer. Each communication edge between two products carries a controlStrength score (a product of accessibility, configuration hygiene, exploit resistance, and hardening — Eq. 3), which discounts the standard EPSS exploitation-probability figure into a context-aware pExploit (Eq. 4). An attackCost term captures adversarial effort from the CVSS vector (Eq. 5), and these combine into a per-edge riskWeight (Eq. 6). Multi-hop attack-path likelihood is the product of pExploit along the path (Eq. 7), and node-level exposure is the weighted sum of incoming risk (Eq. 8). A parallel CONTROLLED_COMMUNICATES_WITH relation re-runs the same math after NIST SP 800-53 / IEC 62443 controls (segmentation, patching, IDS, access control, hardening) are applied, so the framework can directly compare risk before and after mitigation.

A knowledge-enrichment layer uses domain-tuned transformer models — SecureBERT and CySecBERT for embeddings, REBEL for relation extraction — to recover CVE↔CWE and CVE↔ATT&CK links that the public datasets leave sparse, plus a natural-language-to-graph pipeline that turns unstructured advisory text into graph triples.

Results

- Enrichment accuracy: a BERT classifier using CySecBERT embeddings reached 66.47% accuracy predicting missing CWE links; a CyberBERT classifier using SecRoBERTa embeddings reached 98.70% accuracy predicting missing ATT&CK technique links.
- Across 15 simulated multi-stage attack scenarios, enriching the graph with inferred communication links shortened average attack paths by 20–40% (revealing hidden routes an attacker could take), while applying NIST/IEC-aligned controls lengthened average paths by 25–50% (showing measurable segmentation benefit).
- Mean attack-path length moved from ≈4.0 hops (raw graph) to ≈3.5 hops (enriched) to ≈5.5 hops (after controls), with non-overlapping confidence intervals across the three configurations.
- A representative MQTT-broker-to-PLC scenario: enrichment cut the minimum path to a single hop and expanded reachable PLCs from 8 to 9; applying controls cut reachability back to 7 PLCs.
- Centrality analysis (PageRank, betweenness) showed enrichment shifts structural importance toward communication intermediaries like gateways and workstation nodes — useful for identifying real chokepoints rather than just individually risky assets.

Conclusion

BRIDG-ICS combines contextual graph enrichment, inferred relations, and probabilistic attack-path modeling into one framework for reasoning about IT–OT risk propagation in smart manufacturing. Domain-specific embeddings (CySecBERT, SecRoBERTa) filled in missing CVE–CWE–Technique–Tactic links well enough to support graph-based reasoning. Across the 15 evaluated scenarios, enrichment shortened attack paths by 20–40% while NIST-aligned controls lengthened them by roughly 25–50%, and graph analytics surfaced critical assets and cascade-prone communities. The authors flag reliance on synthetic/testbed logs and a fairly generic entity/relation taxonomy as current limitations, and describe plans to add LLM-assisted reasoning and multi-agent, scenario-based simulation in future work.

References

[1] Hassan, M.A. et al. Systematic analysis of risks in industry 5.0 architecture. Applied Sciences 14(4), 1466 (2024)

[2] Sun, X., Song, Y. Unlocking the synergy: increasing productivity through human-AI collaboration in the industry 5.0 era. Computers & Industrial Engineering 200, 110657 (2025)

[3] Tyagi, P. Convergence of IT and OT cybersecurity related challenges and best practices. IJCTT 69(2), 85–92 (2021)

[4] Srinivasan, H., Karimi, M. Threat-based Security Controls to Protect Industrial Control Systems. arXiv:2501.13268 (2025)

[5] Anton, S.D.D. et al. Devil in the Detail: Attack Scenarios in Industrial Applications. arXiv:1905.10292 (2019)

[6] Toward sustainability and resilience with industry 4.0 and industry 5.0. Frontiers in Manufacturing Technology 1, 861656 (2022)

[7] Kaspersky ICS-CERT. A Brief Overview of the Main Incidents in Industrial Cybersecurity Q2 2025 (2025)

[8] Belikovetsky, S. et al. dr0wned: Cyber-physical attack with additive manufacturing. USENIX WOOT (2017)

[9] Alqudhaibi, A. et al. Securing industry 4.0: cybersecurity challenges and strategies for manufacturing. Cyber Security and Applications 3, 100067 (2025)

[10] Miller, T. et al. Looking back to look forward: lessons from cyber-attacks on ICS. Int. J. Critical Infrastructure Protection 35, 100464 (2021)

[11] Anton, S.D.D. et al. The global state of security in ICS. IEEE IoT Journal 8(24), 17525–17540 (2021)

[12] Afenu, D.S. et al. ICS security validation based on MITRE ATT&CK. Electronics 13(5) (2024)

[13] Nankya, M. et al. Securing ICS: components, threats, and ML defense strategies. Sensors 23(21), 8840 (2023)

[14] Al-Sada, B. et al. MITRE ATT&CK: state of the art and way forward. ACM Comput. Surv. 57(1) (2024)

[15] Jadidi, Z., Lu, Y. A threat hunting framework for industrial control systems. IEEE Access 9, 164118–164130 (2021)

[16] MITRE. ATT&CK for Industrial Control Systems (ICS) (2022)

[17] Salek, M.S. et al. An LLM-supported threat modeling framework for transportation CPS. IEEE Access 13, 163046–163070 (2025)

[18] Alnajim, A.M. et al. A comprehensive survey of cybersecurity threats in industrial IoT. Technologies 11(6) (2023)

[19] Badawy, M. et al. Legacy ICS cybersecurity assessment using hybrid threat modeling. Applied Sciences 14(18) (2024)

[20] Alanen, J. et al. Hybrid ontology for safety, security, and dependability risk assessments. Reliability Eng. & System Safety 220, 108270 (2022)

[21] Khalil, S.M. et al. Threat modeling of industrial control systems: a systematic literature review. Computers & Security 136, 103543 (2024)

[22] Cyber Initiative. Automated Construction of Threat Knowledge Graph for ICS Security (2021)

[23] NIST. Guide to Industrial Control Systems (ICS) Security, SP 800-82 Rev. 3 (2023)

[24] ISA/IEC 62443. Industrial Automation and Control Systems Security (2021)

[25] Byres, E. The myths and facts behind cyber security risks for ICS (2004)

[26] Makrakis, G.M. et al. Vulnerabilities and attacks against ICS and critical infrastructures. arXiv:2109.03945 (2021)

[27] Song, H. et al. A security posture assessment of ICS based on evidential reasoning. Sensors 24(22), 7135 (2024)

[28] Kour, R. et al. Cybersecurity for industry 5.0: trends and gaps. Frontiers in Computer Science 6 (2024)

[29] Naik, N. et al. A comparative analysis of threat modelling methods: STRIDE, DREAD, VAST, PASTA, OCTAVE, LINDDUN (2024)

[30] Mavroeidis, V., Bromander, S. Cyber threat intelligence model: taxonomies, sharing standards and ontologies. EISIC (2017)

[31] Hogan, A. et al. Knowledge graphs. ACM Computing Surveys 54(4), 1–37 (2021)

[32] Ji, S. et al. A survey on knowledge graphs: representation, acquisition, applications. IEEE TNNLS (2022)

[33] Branescu, I. et al. Automated mapping of CVEs to MITRE ATT&CK tactics. Information 15(4) (2024)

[34] Gao, P. et al. ThreatKG: an AI-powered system for automated CTI gathering. arXiv:2212.10388 (2024)

[35] Li, Z. et al. AttacKG: constructing technique knowledge graph from CTI reports. arXiv:2111.07093 (2022)

[36] Sun, L. et al. APTKG: constructing threat intelligence KG from open-source APT reports. DSIT (2022)

[37] Zhang, Y. et al. AttacKG+: boosting attack graph construction with LLMs. Computers & Security 150, 104220 (2025)

[38] Sikos, L.F. Cybersecurity knowledge graphs. Knowledge and Information Systems 65(9), 3511–3531 (2023)

[39] Kurniawan, K. et al. The ICS-SEC KG: an integrated cybersecurity resource for ICS (2025)

[40] CISA. Industrial Control Systems Advisories (ICSA) (2024)

[41] Zhao, X. et al. A survey on cybersecurity knowledge graph construction. Computers & Security 136, 103524 (2024)

[42] Huang, L., Xiao, X. CTIKG: LLM-powered knowledge graph construction from CTI. COLM (2024)

[43] Yin, J. et al. A compact vulnerability knowledge graph for risk assessment. ACM TKDD 18(8) (2024)

[44] Høst, A. et al. Constructing a knowledge graph from textual CVE descriptions. NoDaLiDa (2023)

[45] Sarhan, I., Spruit, M. Open-CyKG: an open cyber threat intelligence knowledge graph. Knowledge-Based Systems 233, 107524 (2021)

[46] Mouiche, I., Saad, S. Entity and relation extractions for threat intelligence KGs. Computers & Security 148, 104120 (2025)

[47] Jia, Y. et al. A practical approach to constructing a knowledge graph for cybersecurity. Engineering 4(1), 53–60 (2018)

[48] Falcarin, P., Dainese, F. Building a cybersecurity knowledge graph with CyberGraph. EnCyCriS/SVM (2024)

[49] Zhou, Y. et al. AEKG4APT: an AI-enhanced knowledge graph for APTs with LLM analysis (2025)

[50] Liu, K. et al. Recent progress of using knowledge graph for cybersecurity. Electronics 11(15) (2022)

[51] Fujimoto, M. et al. Efficient ICS risk assessment using the attack path to the critical device. MSIE (2021)

[52] Shen, G. et al. Data-driven cybersecurity knowledge graph construction for ICS security (2020)

[53] Zhang, X. et al. MER-GCN: reasoning about attacking group behaviors using ICS attack KGs. Computers & Security 157, 104558 (2025)

[54–69] Additional ontology, XAI, and GNN references (UCO, STIX, Security Ontology, IOF, ISA/IEC 62443 ontology mapping, HINTI, CSKG4APT, explainable-AI surveys, and MITRE catalog entries — full list available in the source PDF/HTML at arxiv.org/abs/2512.12112).

Reference Paper 2

SkyEye: When Your Vision Reaches Beyond IAM Boundary Scope in AWS Cloud

Nguyen, Ho, To (2025) — arXiv:2507.21094 — Université Libre de Bruxelles / TU Darmstadt

Abstract

As enterprises move workloads to AWS, IAM becomes the primary security boundary, and thoroughly enumerating what a set of credentials can actually do is essential for both defenders and red teams. The authors argue that existing IAM enumeration tools work one credential at a time, which structurally hides permissions and escalation paths that only become visible when multiple obtained credentials are considered together. SkyEye is presented as a cooperative, multi-principal IAM enumeration framework that correlates several AWS credentials at once to build a complete picture of a target's IAM exposure, closing the visibility gap left by single-principal tools.

System Architecture

SkyEye's core contribution is the Cross-Principal IAM Enumeration Model (CPIEM), which runs enumeration across multiple AWS principals simultaneously rather than one at a time. Where a conventional tool can only see what a single credential is authorized to do, CPIEM correlates sessions from several valid credentials at once, surfacing privilege-escalation paths and resource access that only exist because of interaction between principals — for example, where user A alone can't reach a sensitive resource, but user A combined with role B's temporary session can.

Layered on top is the Transitive Cross-Role Enumeration Model (TCREM), which recursively follows AssumeRole chains — tracking not just roles a principal can assume directly, but roles reachable indirectly through one or more intermediate role assumptions. This expands the enumeration scope to the full set of in-scope roles rather than stopping at first-hop assumptions.

Beneath these two models sits a deep enumeration layer that systematically retrieves inline policies, attached managed policies, and group/role memberships for every in-scope principal, plus an inverse enumeration mode (using iam:ListEntitiesForPolicy) that works backward from a policy to find every entity it's attached to. Findings are further mapped onto MITRE ATT&CK for Cloud to give each discovered path a severity level and an associated abuse technique, rather than leaving results as an unranked permission dump.

The paper frames its contribution against seven established tools — PACU, CloudPEASS, enumerate-iam, CloudFox, ScoutSuite, Stratus Red Team, and Cloudsplaining — scored across six capability dimensions: principal scope (single vs. multi), transitive role-chain detection, discovery mechanism (static rules vs. active enumeration vs. fuzzing), temporal/session awareness, ATT&CK mapping, and cross-policy aggregation (identity policies + resource policies + permission boundaries + SCPs unified into one view). SkyEye is positioned as the only one of the eight that scores 'high' across all six dimensions, principally because none of the prior tools correlate multiple principals or resolve transitive role chains.

Results

The evaluation is a scenario-based benchmark: the authors constructed a set of proposed attack/enumeration scenarios, each carrying a weight reflecting its real-world relevance, and scored SkyEye against the six published frameworks above on how many of the scenarios each tool could fully resolve (see the Benchmarking Table in the paper's Chapter 5). The comparative matrix shows SkyEye as the only tool with full transitive role-assumption-chain detection, and the only one combining active API-based enumeration with fuzzing/permutation-style discovery of unknown attack surfaces rather than relying solely on a fixed catalog of known escalation patterns — the limitation the authors identify in every prior tool they reviewed (PACU, CloudPEASS, enumerate-iam, CloudFox, ScoutSuite, Stratus Red Team, Cloudsplaining all rely on single-principal analysis and/or static, pre-catalogued escalation patterns).

Conclusion

The paper's central claim is that single-principal IAM enumeration is a structural blind spot, not just an implementation gap — cross-principal and transitive privilege paths are provably invisible to tools that evaluate one credential's permissions in isolation. SkyEye's CPIEM and TCREM models are proposed as a way to close that gap by correlating multiple credentials and following role-assumption chains recursively. The authors position this as directly relevant to real breach patterns (they cite Capital One 2019 and the LAPSUS$ campaigns as motivating cases) where the exploited path only existed because of interactions across multiple identities that no single-principal tool would have surfaced.

References

This is a 105-page thesis-style paper; the full numbered bibliography runs to several dozen entries covering AWS documentation, breach post-mortems, and academic IAM-security literature. The tool/framework citations that anchor its comparative analysis (Chapter 3) are reproduced below; the complete reference list is available in the source PDF.

[1] Rhino Security Labs. PACU – The AWS Exploitation Framework.

[2] CloudPEASS – Cloud Privilege Escalation Awesome Script Suite.

[3] enumerate-iam – Enumerate AWS IAM Permissions (NCC Group).

[4] CloudFox – Automate Situational Awareness for Cloud Penetration Tests.

[5] ScoutSuite – Multi-Cloud Security Auditing Tool (NCC Group).

[6] Stratus Red Team (Datadog) – Granular, Actionable Adversary Emulation for the Cloud.

[7] Cloudsplaining (Salesforce) – AWS IAM Security Assessment Tool.

[8–19] Capital One breach analysis, LAPSUS$ incident reporting, ENISA Threat Landscape 2024, AWS IAM documentation and service authorization references, and related academic IAM-attack detection papers (full citations in source PDF, arxiv.org/abs/2507.21094).

Reference Paper 3

Fixing Privilege Escalations in Cloud Access Control with MaxSAT and Graph Neural Networks (IAMPERE)

Hu, Wang, Khurshid, McMillan, Tiwari — IEEE/ACM ASE 2023 — DOI 10.1109/ASE56229.2023.00167 — UT Austin

Abstract

IAM misconfigurations that enable privilege escalation (PE) are well studied for detection, but repair — automatically fixing a misconfiguration once found — is comparatively unexplored; the only prior tool, IAM-Deescalate, covers a narrow set of PE patterns. The authors present IAMPERE, which generates an approximately minimal patch for a much broader class of IAM PEs. The repair problem is first formulated as a MaxSAT problem; because MaxSAT solvers alone don't scale well on large configurations, a graph neural network is trained to propose a small (but not necessarily minimal) intermediate patch, which a MaxSAT solver then refines down to a near-minimal final patch. Across synthetic and real-world misconfigurations, IAMPERE repairs substantially more cases than IAM-Deescalate, with markedly smaller patches.

System Architecture

IAM configurations are first converted into a permission flow graph (PFG) — a reduced, semantic representation where nodes are entities (users, roles, services, groups) and edges represent both direct permission assignments and indirect 'permission flows' (e.g., a user inheriting all of a group's permissions). This graph is reduced to only the entities and flows reachable from a compromised entity, since anything else can't be exploited.

Safety verification models the attacker's behavior as a finite-state machine over this graph, alternating between Permission-based Transitions (applying a permission) and Flow-based Transitions (permissions propagating through enabled flows), using fixed-point iteration to prove whether a configuration is safe or find a bounded counterexample.

The repair problem itself is encoded as MaxSAT: hard constraints enforce that the repaired configuration is verifiably safe (via bounded model checking), and soft constraints represent each candidate repair operation (revoking a permission flow, or revoking a permission assignment) — minimizing violated soft constraints is equivalent to finding a minimum patch.

Because MaxSAT alone doesn't scale to large real-world configurations, a GNN (six Graph Attention Network layers with normalization and skip connections) is trained on the PFG's node/edge structure to predict, for every candidate repair operation, the probability it appears in a minimal patch. The top-ranked operations form an 'intermediate patch,' which is then handed to the MaxSAT solver to refine into a smaller, near-minimal final patch — dramatically shrinking the solver's search space compared to solving from scratch.

Results

- On Test-A (1,000 synthetic large-scale misconfigurations, up to 315 entities and ~11,700 permissions): IAMPERE repaired 57.6% of cases within the 600-second time limit, versus 21.6% for IAM-Deescalate — more than double the repair rate.
- An ablation (IAMPERE-MO, MaxSAT only, no GNN) repaired only 33.5% — confirming the GNN-guided search is doing substantial work, not just the MaxSAT solver alone.
- For patches under 20% of the maximum possible patch size (i.e., small, targeted fixes), IAMPERE produced 145 more such patches than IAM-Deescalate.
- On Test-B (two real-world misconfigurations from a US cloud security startup, both containing 10+ transitive PEs with path lengths ≥5): IAMPERE successfully repaired both within a 2-hour limit; IAM-Deescalate failed on both.
- On Test-C (31 misconfigurations from the public IAM-Vulnerable benchmark): IAMPERE and its variants repaired all 31 with optimal patches in under 5 seconds each; IAM-Deescalate repaired 24 of 31.

Conclusion

IAMPERE's repairs are sound — any patch it produces is guaranteed to eliminate the identified privilege escalations — though not guaranteed minimal, since the GNN-guided search trades optimality for scalability. The authors frame the GNN-assisted MaxSAT pattern as a contribution beyond IAM repair specifically, potentially applicable to other MaxSAT-heavy domains like scheduling, hardware verification, and bioinformatics. IAMPERE and its dataset are open-sourced at github.com/githubhuyang/iampere.

References

[1] AWS. AWS Identity and Access Management (IAM). aws.amazon.com/iam/ (2023)

[2] Capital One. Information on the Capital One Cyber Incident (2022)

[3] Pernet, C. Research reveals that IAM is too often permissive and misconfigured. TechRepublic (2021)

[4] Quist, N. Unit 42 Cloud Threat Report: cloud security weakens as IAM goes unsecured (2021)

[5] Morag, A. Cloud misconfigurations: the hidden but preventable threat. Infosecurity Magazine (2021)

[6] Cybersecurity Insiders. Cloud Security Report (2021)

[7] DivvyCloud. 2020 Cloud Misconfigurations Report (2020)

[8] Gietzen, S. AWS IAM Privilege Escalation – Methods and Mitigation. Rhino Security Labs (2018)

[9] Kleijn, G. Well, that escalated quickly: privilege escalation in AWS. Bishop Fox (2022)

[10] Kedrosky, E. Achieving AWS least privilege: understanding privilege escalation. Sonrai Security (2022)

[11] Rhino Security Labs. PACU: the open source AWS exploitation framework (2022)

[12] Salesforce. Cloudsplaining (2022)

[13] NCC Group. Principal Mapper (2023)

[14] WithSecure Labs. awspx – graph-based tool for visualizing AWS access relationships (2022)

[15] Hu, Y., Wang, W., Khurshid, S., Tiwari, M. Interactive greybox penetration testing on cloud access control with IAM modeling and deep RL (2023) [TAC]

[16] Shevrin, I., Margalit, O. Detecting multi-step IAM attacks in AWS environments via model checking. USENIX Security 32 (2023)

[17] AWS. Service Authorization Reference (2023)

[18] AWS. Security best practices in IAM (2023)

[19] Zscaler. Anatomy of a cloud breach: how 100 million credit card numbers were exposed (2021)

[20] Chen, J. IAM-Deescalate: reducing the risk of privilege escalation. Unit 42 (2022)

[21] Wu, Z. et al. A comprehensive survey on graph neural networks. IEEE TNNLS 32(1), 4–24 (2020)

[22] Zhou, J. et al. Graph neural networks: a review of methods and applications. AI Open 1, 57–81 (2020)

[23] Gilmer, J. et al. Neural message passing for quantum chemistry. ICML (2017)

[24] Kipf, T.N., Welling, M. Semi-supervised classification with graph convolutional networks. arXiv:1609.02907 (2016)

[25] Hamilton, W.L. et al. Inductive representation learning on large graphs. NeurIPS (2017)

[26] McMillan, K.L. Symbolic model checking. Springer (1993)

[27] Brody, S. et al. How attentive are graph attention networks? arXiv:2105.14491 (2021)

[28] Cai, T. et al. GraphNorm: accelerating GNN training. ICML (2021)

[29] You, J. et al. Design space for graph neural networks. NeurIPS 33 (2020)

[30] Bacchus, F. et al. MaxSAT Evaluation 2022 (2022)

[31] Bishop Fox. IAM Vulnerable – assessing the AWS assessment tools (2021)

[32] Bacchus, F. et al. MaxSAT Evaluation 2021 (2021)

[33] Bacchus, F. et al. MaxSAT Evaluation 2020 (2020)

[34] Loshchilov, I., Hutter, F. Decoupled weight decay regularization. arXiv:1711.05101 (2017)

[35] Bouchet, M. et al. Block Public Access: trust safety verification of access control policies. ESEC/FSE (2020)

[36] Backes, J. et al. Semantic-based automated reasoning for AWS access policies using SMT (ZELKOVA). FMCAD (2018)

[37–46] Additional MaxSAT-application references spanning planning/scheduling, hardware/software verification and repair, and bioinformatics (full list in source PDF).

Reference Paper 4

CloudLens: Modeling and Detecting Cloud Security Vulnerabilities

Kazdagli, Tiwari, Kumar (2024) — arXiv:2402.10985 — UT Austin / Symmetry Systems / Singapore Management University

Abstract

The authors present CloudLens, a formal relational model for representing cloud IAM — users, roles, groups, datastores, and the permissions connecting them — and CloudExploit, a planning-based tool built on top of it that automatically searches for exploitable attack sequences (privilege escalation, ransomware, data exfiltration, and more) rather than just flagging individual risky permissions. Tested on 14 real commercial AWS configurations, CloudExploit found a broader range of vulnerabilities than existing industry tools such as PMapper.

System Architecture

CloudLens represents the entire IAM state as a set of simple relational tuples: ⟨id1, perm, id2⟩ for permissions one identity has over another (e.g., assumeRole), ⟨id, perm, ds⟩ for permissions over a datastore, and two 4-tuple variants that represent permissions to grant other permissions (capturing indirect, delegated privilege). This tuple format is deliberately simple so it generalizes across AWS, GCP, and Azure and so IAM policies can be parsed directly into it from JSON.

On top of this representation, the paper casts IAM attack discovery as an AI planning problem using PDDL (Planning Domain Definition Language). Three action types drive the search: permissionFlow (permissions propagating from one identity to another via role assumption, group membership, or attached policy), addTuples (a compromised identity directly modifying the IAM state, e.g., attaching a new policy to itself), and enableAttack (a terminal action representing a successful attack of a given type — ransomware, data exfiltration, privilege escalation, persistence, lateral movement, or impact). The planner (Fast Downward) searches for the lowest-cost sequence of actions that reaches an enableAttack state from a given compromised identity, which directly corresponds to the shortest realistic attack path.

For scalability on large real-world tenants (some with over a million permissions), the tool partitions accounts into non-overlapping groups of users reachable via breadth-first search along permission edges, rather than solving the whole tenant graph at once.

Results

- Verification against the public IAM-Vulnerable benchmark (31 scenarios): CloudExploit correctly detected all 19 scenarios within its modeled AWS service scope (IAM, EC2, S3, Lambda, STS, SSM); the other 12 involved services deliberately left out of scope for solver performance reasons.
- Across 14 real commercial cloud configurations (hundreds to thousands of identities each), CloudExploit identified 507 non-admin users exploitable for at least one of six attack types.
- Nearly every environment had at least one user capable of a full ransomware attack, traceable to just two commonly over-granted permissions (s3:PutObject and kms:CreateKey) combined with disabled S3 versioning/MFA-delete protections.
- Three organizations were found vulnerable to privilege escalation despite this being unexpected going in; one dataset had widespread accidental exposure of a sensitive S3 bucket originally intended only for internal software-development use.
- Attack path lengths were mostly short (2–3 actions), meaning many compromised users already held the permissions needed to attack directly; longer 4+ step paths typically involved chained role assumptions or group-based permission inheritance.
- Head-to-head against PMapper (a leading open-source privilege-escalation-only tool): CloudExploit detected far more vulnerable users across all 14 datasets, since PMapper can only find privilege-escalation paths and has no visibility into datastore-involving attacks like ransomware or exfiltration.

Conclusion

The paper's core argument is that treating IAM security as a planning problem — rather than a static rule-matching problem — lets a single formal model surface a genuinely broader class of attacks (six types, versus privilege-escalation-only in prior tools like PMapper) while remaining explainable, since the tuple representation is just a flat text format an IT admin can read directly. The authors position CloudExploit as a 'security co-pilot' for IT administrators rather than a fully automated fix-it tool, since real repair requires business context (which access is actually needed) that the model doesn't

have. Planned future work includes minimal diverse-plan generation to identify the weakest precondition(s) enabling an attack, to guide targeted remediation.

References

[1] Bishop Fox. IAM Vulnerable – Assessing the AWS Assessment Tools (2021)

[2] CPDDL planning library (2024)

[3] Amazon AWS. AWS Documentation (2023)

[4] AWS. Shared Responsibility Model (2023)

[5] Amazon AWS. Types of access control (2024)

[6] Backes, J. et al. Semantic-based Automated Reasoning for AWS Access Policies using SMT (ZELKOVA). FMCAD (2018)

[7] Batra, G. et al. Deploying ABAC policies using RBAC systems. Journal of Computer Security 27, 1–24 (2019)

[8] Blum, A.L., Furst, M.L. Fast planning through planning graph analysis. Artificial Intelligence 90(1), 281–300 (1997)

[9] Boddy, M. et al. Course of Action Generation for Cyber Security Using Classical Planning. ICAPS (2005)

[10] CSA. Top Threats to Cloud Computing: Pandemic 11 Deep Dive (2023)

[11] Bishop Fox. IAM Vulnerable (GitHub) (2021)

[12] Bishop Fox (company site) (2023)

[13] Gietzen, S. AWS IAM Privilege Escalation – Methods and Mitigation. Rhino Security Labs (2021)

[14] Green, A. et al. PDDL – The Planning Domain Definition Language (2023)

[15] Helmert, M. The Fast Downward Planning System. JAIR 26(1), 191–246 (2006)

[16] Hoffmann, J. Simulated Penetration Testing: from Dijkstra to Turing Test++. ICAPS (2015)

[17] Holmes, A. 533 million Facebook users' data leaked online. Business Insider (2021)

[18] Hu, Y. et al. Fixing Privilege Escalations in Cloud Access Control with MaxSAT and GNNs [IAMPERE]. ASE (2023)

[19] Jayaraman, K. et al. Validating Datacenters at Scale. ACM SIGCOMM (2019)

[20] Katz, M., Sohrabi, S. Reshaping Diverse Planning. AAAI (2020)

[21] Kazdagli, M. et al. Using Constraint Programming and Graph Representation Learning for Interpretable Cloud Security Policies. IJCAI, 1850–1858 (2022)

[22] Kuenzli, S. Why are good AWS security policies so difficult? K9 Security (2020)

[23] Rhino Security Labs. S3 Ransomware Part 1: Attack Vector (2019)

[24] Lippmann, R., Ingols, K. An Annotated Review of Past Papers on Attack Graphs (2005)

[25] Marks, G. A LinkedIn 'Breach' Exposes 92% of Users. Forbes (2021)

[26] McDermott, D. et al. PDDL – The Planning Domain Definition Language. AIPS-98 (1998)

[27] MITRE. Caldera (2024)

[28] NCC Group. Principal Mapper (PMapper) (2023)

[29] Nguyen, T.H. et al. Multistage Attack Graph Security Games. Security and Communication Networks (2018)

[30] Obes, J.L. et al. Attack Planning in the Real World. SecArt (2010)

[31] Capital One. Information on the Capital One Cyber Incident (2022)

[32] Pang, R. et al. Zanzibar: Google's Consistent, Global Authorization System. USENIX ATC (2019)

[33] Parkinson, S., Khan, S. A Survey on Empirical Security Analysis of Access-control Systems. ACM Comput. Surv. (2022)

[34] Parkinson, S. et al. Automated Planning for Administrating Role-Based Access Control (2020)

[35] Pernet, C. Research reveals IAM is too often permissive and misconfigured. TechRepublic (2021)

[36] Salesforce. Policy Sentry: IAM Least Privilege Policy Generator (2023)

[37] Sarraute, C. et al. POMDPs Make Better Hackers: Accounting for Uncertainty in Penetration Testing. AAAI (2012)

[38] Sasturkar, A. et al. Policy analysis for administrative role-based access control. CSFW (2006)

[39] Scroxton, A. Leaky AWS S3 bucket once again at centre of data breach. Computer Weekly (2020)

[40] Unit42. IAM Your Defense Against Cloud Threats. Palo Alto Networks (2022)

[41] Wang, L. et al. Using attack graphs for correlating, hypothesizing, and predicting intrusion alerts. Computer Communications 29, 2917–2933 (2006)

Reference Paper 5

Behind the Scenes of Attack Graphs: Vulnerable Network Generator for In-Depth Experimental Evaluation of Attack Graph Scalability

Palma, Bonomi — Computers & Security 157 (2025) 104576 — DOI 10.1016/j.cose.2025.104576 — Sapienza University of Rome — Open access (CC BY-NC-ND)

Abstract

Attack graphs are widely used for cyber risk assessment, but generating them at scale is combinatorially expensive, and most prior scalability research varies only network size, ignoring other factors like the number and diversity of vulnerabilities. The authors introduce a vulnerable-network generator that produces realistic reachability graphs paired with vulnerability inventories, validated against three real (anonymized) networks. Using this generator, they build an analytical framework and run an in-depth experimental evaluation of the three foundational attack-graph algorithms — TVA, NetSPA, and MulVAL — identifying which parameters actually drive scalability degradation, and show their findings extend to more recent distributed attack-graph approaches as well.

System Architecture

The paper's main artifact is a synthetic-but-realistic dataset generator. It varies five parameters independently: number of hosts (10–1000), number of vulnerabilities per host (10–500, drawn from real CPE/CVE data rather than randomly), network topology (mesh, star, ring, Erdős–Rényi random, power-law, and a custom four-subnet LAN-based layout mimicking a DMZ/admin/internal-LAN structure), vulnerability distribution across hosts (uniform, Pareto, binomial, Poisson), and vulnerability diversity (0% = every host shares identical vulnerabilities, up to 100% = no two hosts share any). Combined, this produces 24,000 distinct 'vulnerable networks,' each validated for realism against three real organizational networks (a university IT department, a hospital lab, and an automotive IoT/energy-provider testbed) using graph-edit-distance and chi-squared similarity tests — the synthetic generator matched real-network topology with 66–93% similarity and vulnerability distribution with 71–92% similarity.

On top of this dataset sits an analytical framework that builds all three classic attack-graph models (TVA's Exploit Dependency Graph, NetSPA's Multiple Prerequisite Graph, and MulVAL's Logical Attack Graph) from the same input, then measures both space complexity (node/edge counts, graph density, degree distribution, number of connected components, closeness centrality) and time complexity (AG generation time, and separately, attack-path computation time as the number of attack sources/targets scales from 1 to 50).

Results

- RQ1 — Vulnerabilities matter as much as hosts: generation time grows exponentially in both dimensions independently, and roughly doubly-exponentially when both scale together; even a modest 250-host network stays fast if vulnerabilities per host are low (~10), showing host count alone is a poor scalability predictor.
- RQ2 — Vulnerability diversity and topology each swing edge count (and thus cost) by up to 6×: for the topological models (TVA, NetSPA), going from 0% to 100% vulnerability diversity roughly doubled to sextupled the number of graph edges; for MulVAL specifically, topology mattered more than diversity, with mesh topology generating over 6× the edges of a tree topology.
- RQ3 — Where vulnerabilities sit matters, not just how many: a centrality-based Pareto distribution (vulnerabilities concentrated on the most central hosts) computed attack paths in ~0.5–1.5 hours for NetSPA/TVA, while uniform or peripheral-based distributions were impractical (>2 hours) under otherwise identical settings.
- RQ4 — Attack-path computation (not just graph generation) degrades exponentially with hosts, vulnerabilities, and diversity together — described as effectively triple-exponential; beyond ~150 hosts and 150 vulnerabilities per host, path computation became impractical for all three models under existing solutions.
- RQ5 — Distribution and diversity interact: protecting peripheral hosts (concentrating any residual vulnerability risk toward the network's center) kept path computation tractable across the full diversity range, while the reverse (peripheral-based Pareto) became impractical once diversity exceeded roughly 0.5.
- RQ6 — The findings held for a representative modern distributed attack-graph algorithm as well, though distributed approaches showed their own quirk: they were actually slower on small networks (communication/merge overhead dominates) and only paid off at scale — and interestingly, that overhead pattern disappeared once vulnerability diversity was high.
- On three real (anonymized) networks, NetSPA consistently outperformed TVA and MulVAL on attack-path computation time thanks to lower graph centrality variance, but even NetSPA became impractical (>3 hours) once

source/target counts grew past roughly 10×3 — meaning real-world usage is realistically limited to small, targeted source/target sets rather than full-network analysis.

Conclusion

Network size alone is not the dominant driver of attack-graph scalability problems — vulnerability count affects generation time comparably to host count, vulnerability diversity and network topology can each swing graph complexity by up to 6×, and where vulnerabilities are concentrated (central vs. peripheral hosts) has a comparably large effect on whether attack-path computation stays tractable. These conclusions held not just for the three foundational algorithms (TVA, NetSPA, MulVAL) but also for a modern distributed attack-graph generation approach tested for comparison. The authors argue this means prior scalability studies that vary only host count likely understate real-world scalability problems, and they release both the vulnerable-network generator and the analytical framework as open-source tools (github.com/ds-square/vuln-net-ag) to support more comprehensive benchmarking going forward. Planned future work includes deriving a predictive theoretical scalability model from the experimental data, and designing a topology/diversity-aware scalable attack-graph generation algorithm informed by this paper's results.

References

Full reference list (101 entries) is provided in the source PDF (Computers & Security 157 (2025) 104576, pp. 15–17). Key citations anchoring the three foundational algorithms: Jajodia, S., Noel, S., O'Berry, B. Topological analysis of network attack vulnerability [TVA] (2009) · Ingols, K., Lippmann, R., Piwowarski, K. Practical attack graph generation for network defense [NetSPA]. ACSAC (2006) · Ou, X., Govindavajhala, S., Appel, A.W. MulVAL: a logic-based network security analyzer. USENIX Security (2005).

Other load-bearing citations: Kaynar, K. A taxonomy for attack graph generation and usage in network security. J. Inf. Secur. Appl. 29, 27–56 (2016) · Kaynar, K., Sivrikaya, F. Distributed attack graph generation. IEEE TDSC 13(5), 519–532 (2016) · Sabur, A. et al. Toward scalable graph-based security analysis for cloud networks. Computer Networks 206, 108795 (2022) · Palma, A., Angelini, M. It is time to steer: a scalable framework for analysis-driven attack graph generation. ESORICS (2024) · Palma, A., Bonomi, S. A workflow for distributed and resilient attack graph generation. DSN-S (2023) · Zenitani, K. Attack graph analysis: an explanatory guide. Computers & Security (2022).